

RAM-Forensics: Detecting ACPI Rootkits

2021-11-24

Dr. Michael Denzel

> whoami

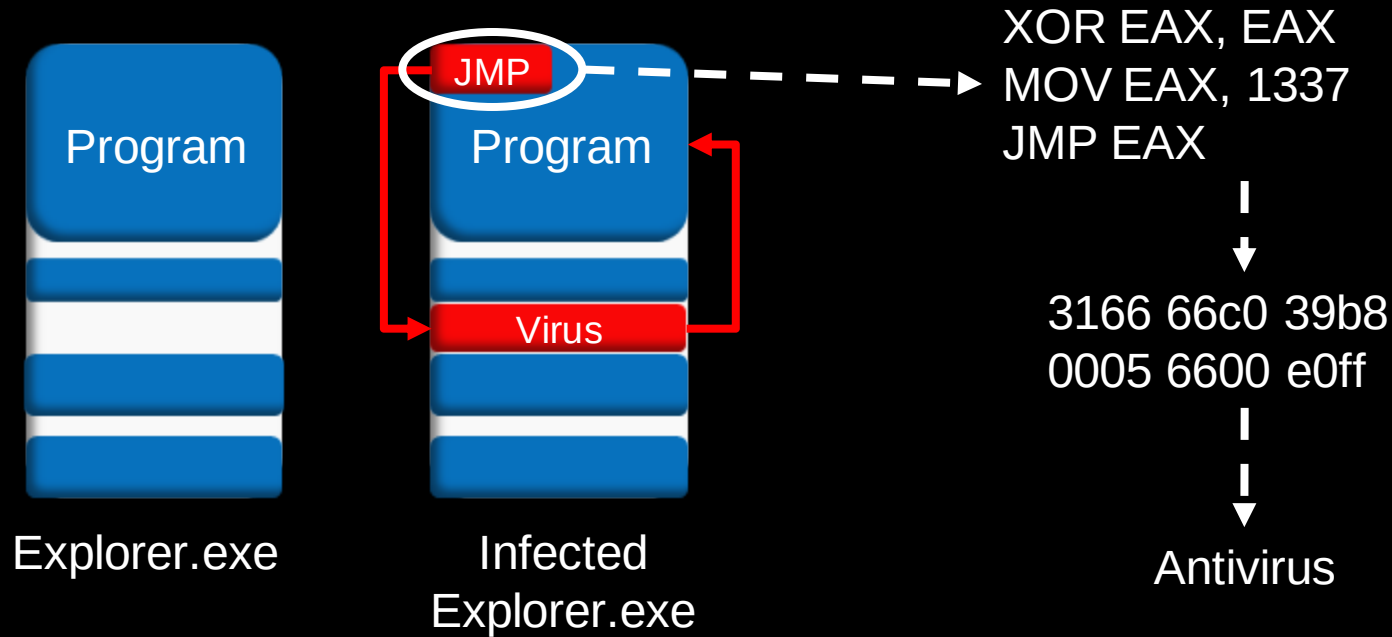
Dr. Michael Denzel

PhD: „Malware tolerance – Distributing trust over multiple devices“

Experience:

- SOC Analyst
 - IT-Security Consultant (building SOCs, DFIR, Pentests)
 - Incident Responder @Airbus CyberSecurity
- } More Blue-Teamer

File-based Malware



File-less Malware

- File-less = no files
- File-less $\neq$ RAM-only
- often: Powershell
- Frameworks: Powershell-Empire, PowerSploit

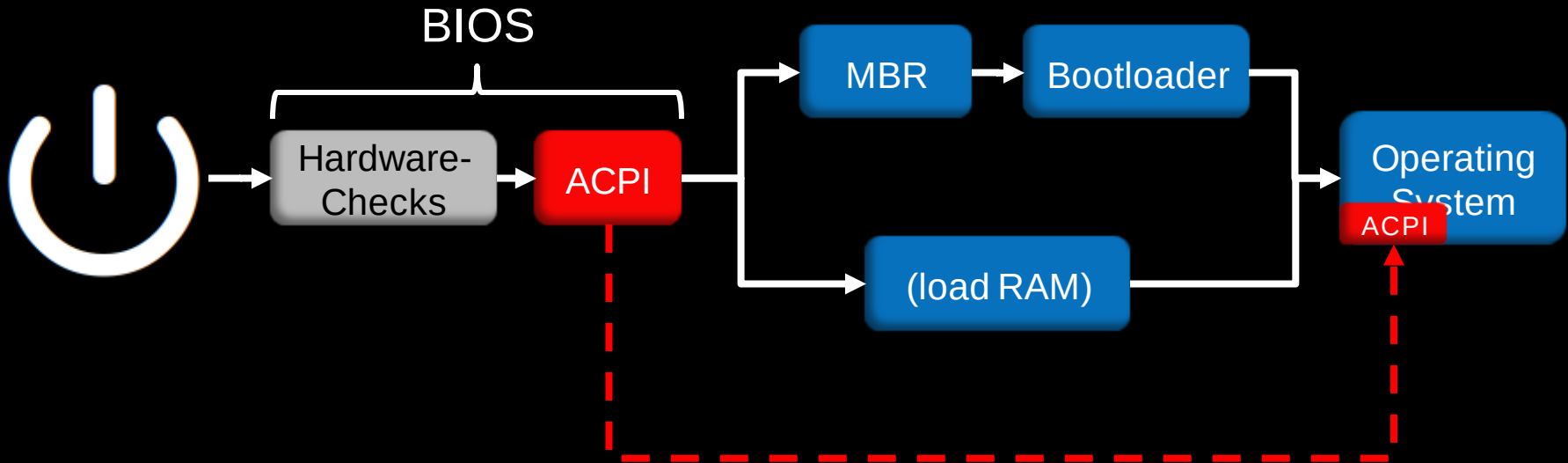
Process Hollowing

1. Start a copy of legitimate process
 2. Suspend process
 3. Deallocate (unmap) and replace code
 4. Resume process
- Looks legitimate: name, path, commandline (e.g. Stuxnet)

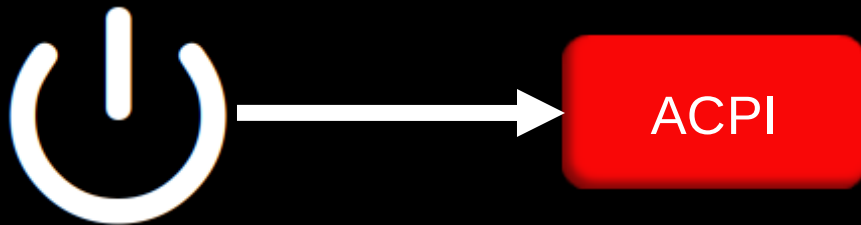
Rootkits

- root + toolkit = software for longterm root/admin access
- Focus: Stealth
- Different levels: user-mode rootkit, kernel rootkit, etc.
- Often paired with backdoor

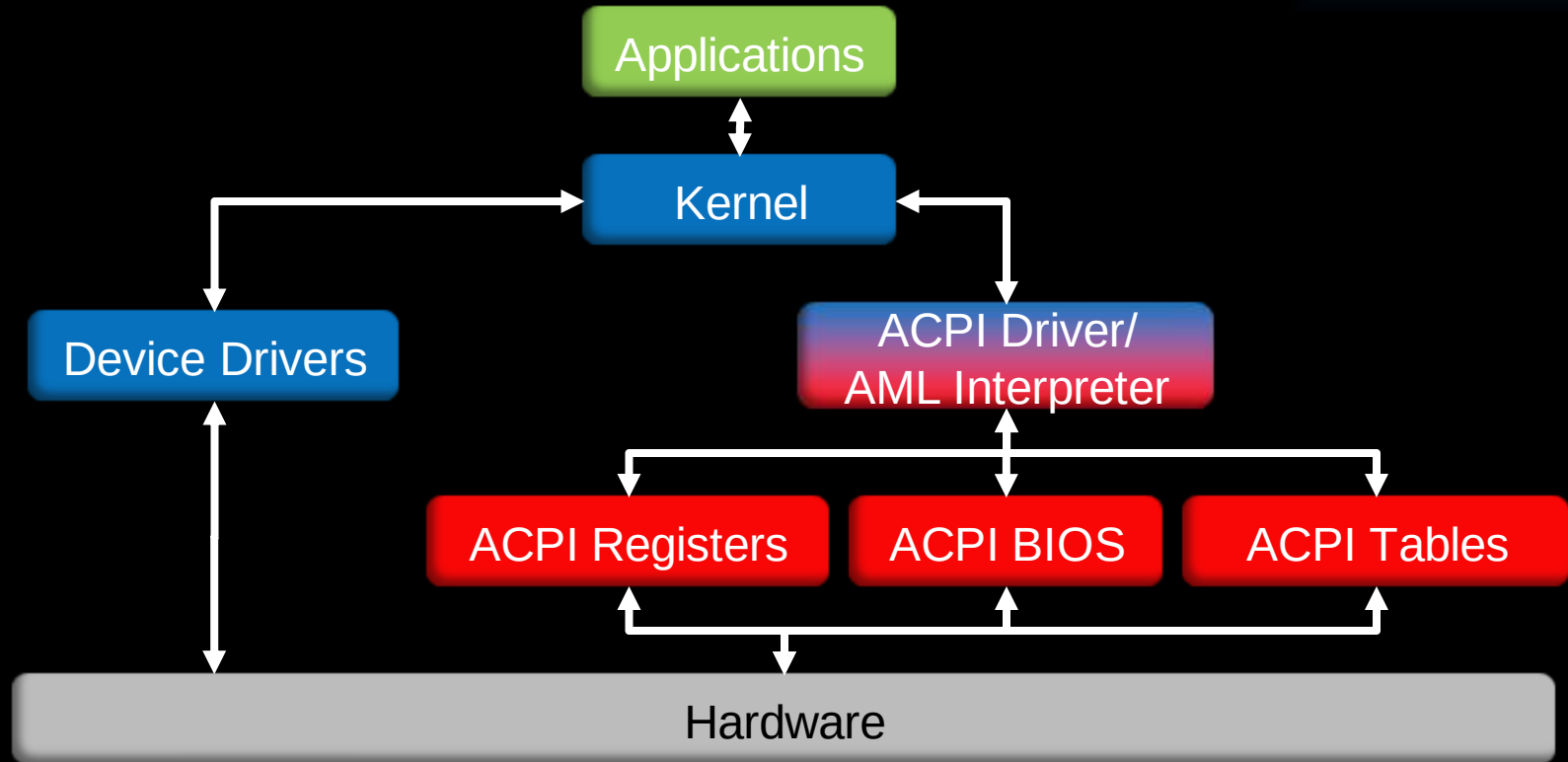
Advanced Configuration and Power Interface (ACPI)



ACPI



ACPI Components



ACPI Tables

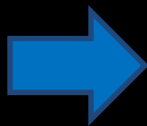
- RSDP „RSD PTR“
 - RSDT / XSDT
 - FADT
 - DSDT (cannot be unloaded)
 - FACS
 - (optional) SSDT
 - (optional) OEMx
 - ... (many more)

ACPI Code: AML/ASL

DSDT.aml

00001510:	4241 5430 085f 4849 440c 41d0 0c0a 085f	BAT0. HID.A....
00001520:	5549 4400 101f 5c5f 4750 4514 185f 4c30	UID... _GPE... L0
00001530:	3000 865c 2f03 5f53 425f 5043 4930 4241	0... _SB.PCI0.BA
00001540:	5430 0a81 5b80 4342 4154 010b 4040 0a08	T0... [.CBAT... @...
00001550:	5b81 1043 4241 5403 4944 5830 2044 4154	[...CBAT.IDX0 DAT
00001560:	3020 5b86 4605 4944 5830 4441 5430 0353	0 [...]F.IDX0DAT0.S

ACPI Machine Language (AML)



DSDT.dsl

```
Device (BAT0)
{
    Name (_HID, EisaId ("PNP0C0A"))
    Name (_UID, Zero)
    Scope (\_GPE)
    {
        Method (_L00, 0, NotSerialized)
        {
            Notify (\_SB.PCI0.BAT0, 0x81)
        }
    }

    OperationRegion (CBAT, SystemIO, 0x4040, 0x08)
    Field (CBAT, DWordAcc, NoLock, Preserve)
    {
        IDX0,    32,
        DAT0,    32
    }
}
```

Disassembled ACPI Source Language (DSL)

ACPI Rootkit

- John Heasman 2006 [1]
- Idea:
 - OperationRegion + SystemMemory + Store
 - Use functions being called at runtime
 - e.g. temperature control

ACPI Rootkit

- PoC: Overwrite unused syscall (sys_ni_syscall) to elevate privileges

```
OperationRegion(NISC, SystemMemory, 0x12BAE0, 0x40)
Field(NISC, AnyAcc, NoLock, Preserve)
{
    NICD, 0x40
}
Store(Buffer () {0xFF, 0xD3, 0xC3, 0x90, 0x90, 0x90, 0x90, 0x90}, NICD)
```

```
{ call ebx; retn; nop; nop; nop; nop; nop }
```

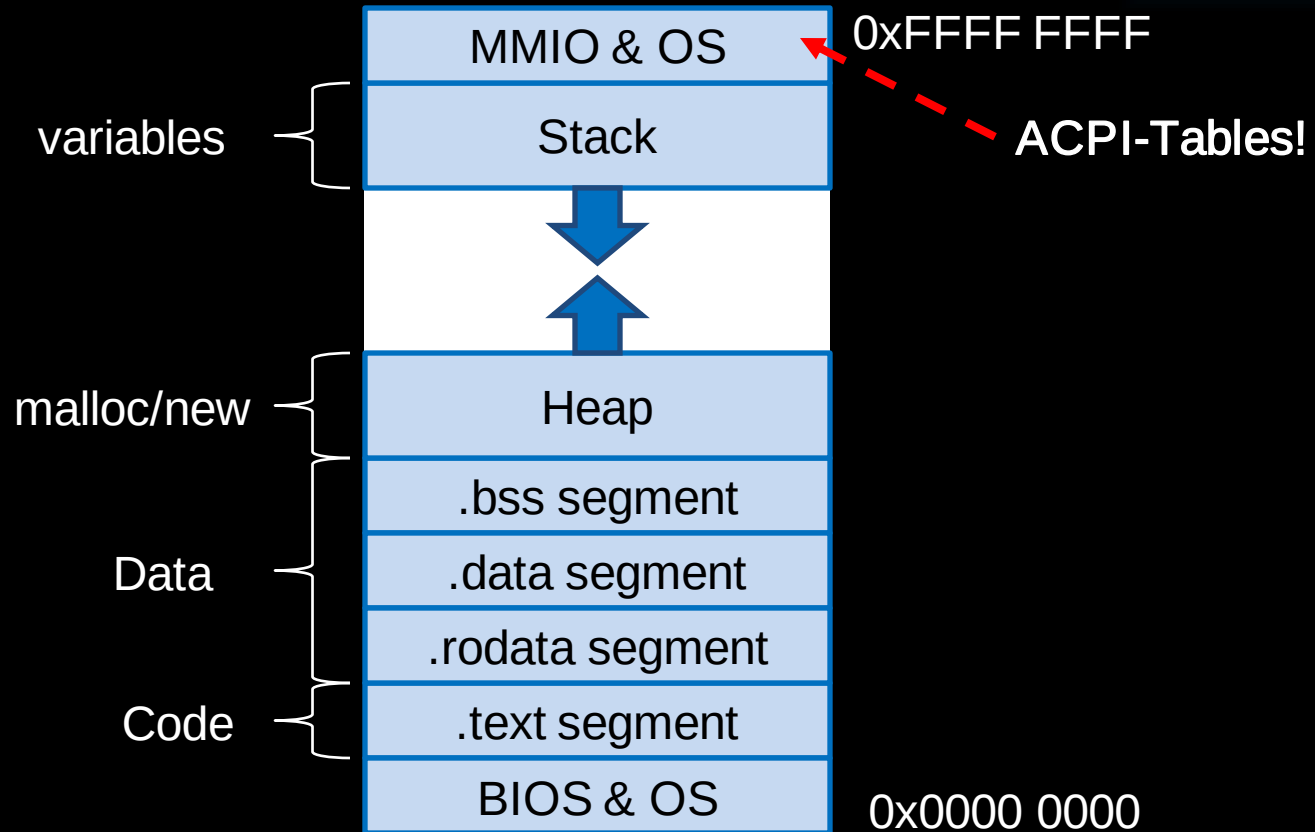
Source [1]

- sys_ni_syscall(evilfunction);

RAM Forensics

- Running processes: RAM → Cache → CPU
- Operating System structures (e.g. `_EPROCESS` list)
- Forensics: lie-detection („screen“ vs HDD/RAM)

RAM Basics



volatility

- by: Michael Cohen
- open-source RAM analysis framework
- commandline only
- Volatility2 (python2) vs Volatility3 („beta“, python3)

volatility Basics

```
$ python vol.py -f ~/Desktop/win7_trial_64bit.raw --profile=Win7SP0x64 pslist
```

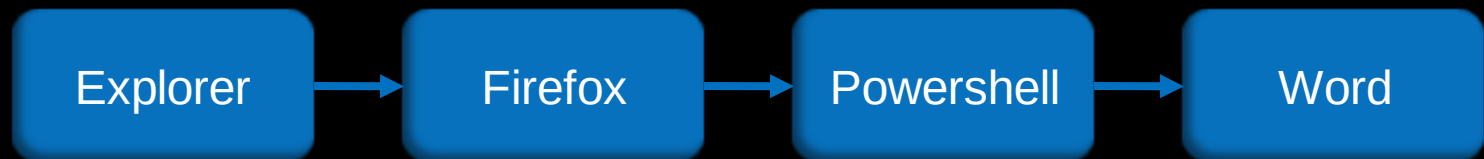
```
Volatility Foundation Volatility Framework 2.4
```

Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start	Exi
0xfffffa80004b09e0	System	4	0	78	489	-----	0	2012-02-22 19:58:20	
0xfffffa8000ce97f0	smss.exe	208	4	2	29	-----	0	2012-02-22 19:58:20	
0xfffffa8000c006c0	csrss.exe	296	288	9	385	0	0	2012-02-22 19:58:24	

Source [2]

volatility Basics

`_EPROCESS` list of operating system:

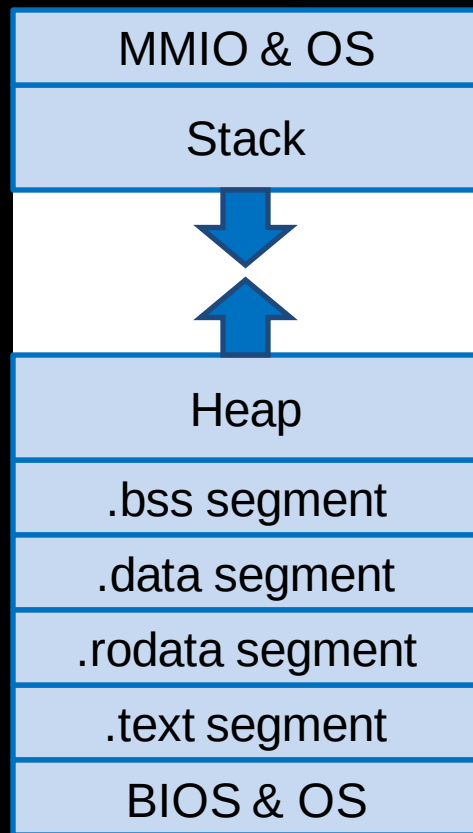


Advanced volatility

Fileless malware:

- `volatility malfind`
- `.text` segment read-only and execute (R-X)
- Injection → write-able (-W-)
- Malfind: find code-pages with WX

Code



Advanced volatility

Process Hollowing:

- Suspend + Unmap + Replace + Start
- volatility ldrmodules

```
$ python vol.py -f ~/Desktop/win7_trial_64bit.raw --profile=Win7SP0x64 ldrmodules
Volatility Foundation Volatility Framework 2.4
```

Pid	Process	Base	InLoad	InInit	InMem	MappedPath
208	smss.exe	0x0000000047a90000	True	False	True	\Windows\System32\smss.exe
296	csrss.exe	0x0000000049700000	True	False	True	\Windows\System32\csrss.exe
344	csrss.exe	0x0000000000390000	False	False	False	\Windows\Fonts\vgasys.fon
344	csrss.exe	0x00000000007a0000	False	False	False	\Windows\Fonts\vgaoem.fon

Source [2]

Advanced volatility

Process Hollowing:

- Suspend + **Unmap** + Replace + Start
- volatility ldrmodules

```
$ python vol.py -f ~/Desktop/win7_trial_64bit.raw --profile=Win7SP0x64 ldrmodules
```

Volatility Foundation Volatility Framework 2.4

Pid	Process	Base	InLoad	InInit	InMem	MappedPath
208	smss.exe	0x0000000047a90000	True	False	True	\Windows\System32\smss.exe
296	csrss.exe	0x0000000049700000	True	False	True	\Windows\System32\csrss.exe
344	csrss.exe	0x0000000000390000	False	False	False	\Windows\Fonts\vgasys.fon
344	csrss.exe	0x00000000007a0000	False	False	False	\Windows\Fonts\vgaoem.fon

Source [2]

volatility

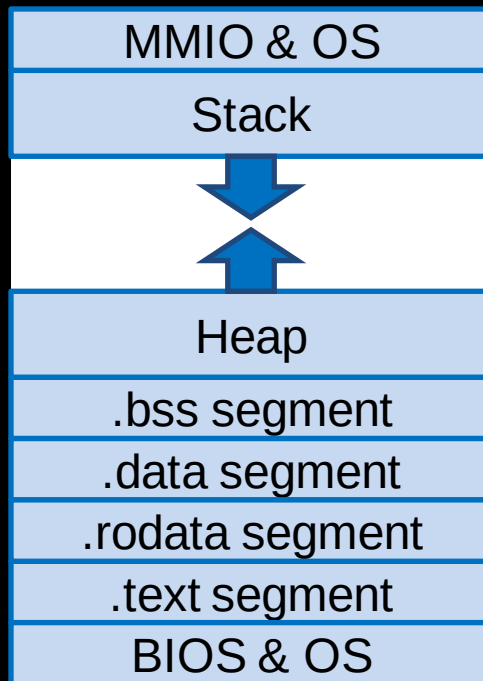
Live-Demo

Advanced volatility

- Fileless Malware: volatility malfind
- Process Hollowing: volatility ldrmodules
- ACPI rootkit: ?

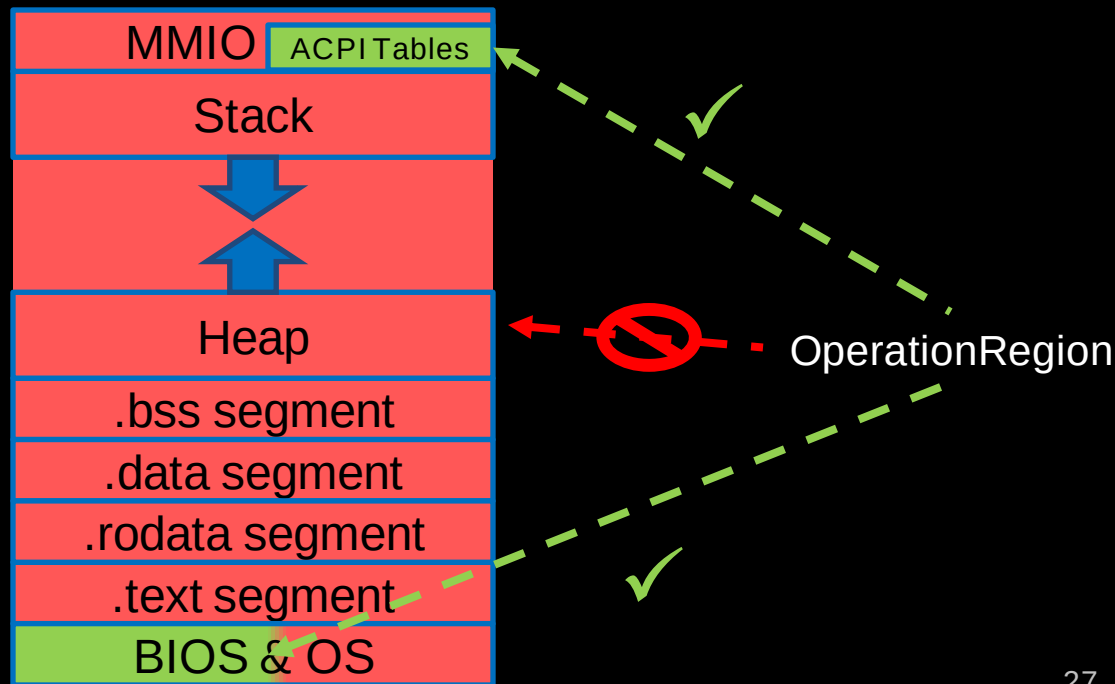
ACPI Rootkit Detection

- What does ACPI not know normally?



ACPI Rootkit Detection

- What does ACPI not know normally?



ACPI Rootkit Detection Tool

```
ctf@localhost ~/p/ACPI-rootkit-scan (master)> python2 ~/progs/volatility/vol.py --plugins=. -f ../Malware/cridex.vmem --profile=WinXPSP2x86 scanacpitables --dump
Volatility Foundation Volatility Framework 2.6.1
```

```
table column "Rootkit?" may have values (seems ok/unknown/suspicious/CRITICAL)
```

File	Function	Rootkit?

WARNING : volatility.debug : FADT.X_FirmwareControl and FADT.FirmwareControl set! According to ACPI documentation this is not allowed! - Trying FirmwareControl		
0x000f6b50/DSDT.dsl	OperationRegion (OEMD, SystemMemory, 0x1FEFFEB9, 0x00000034)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (REGS, PCI_Config, 0x50, 0x30)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (RE00, PCI_Config, 0xD8, 0x04)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (GEN, PCI_Config, 0xB0, 0x04)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (PIRX, PCI_Config, 0x60, 0x04)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (PCI, PCI_Config, 0x40, 0x60)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (MREG, SystemMemory, MBAS (Arg0), 0x10)	suspicious
0x000f6b50/DSDT.dsl	OperationRegion (EICH, SystemMemory, (ECFG + 0x4000), 0x4000)	suspicious
0x000f6b50/DSDT.dsl	OperationRegion (SMIO, SystemIO, 0x000000CF0, 0x00000002)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (SMII, SystemMemory, 0x1FEFFEBD, 0x00000090)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (EREG, SystemMemory, ECFG, 0x4000)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (CREG, SystemMemory, Local1, 0x10)	unknown
0x000f6b50/DSDT.dsl	OperationRegion (CREG, SystemMemory, Local1, 0x01)	unknown
0x000f6b50/DSDT.dsl	OperationRegion (RE01, PCI_Config, 0x40, 0x04)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (RE02, PCI_Config, 0xC4, 0x04)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (REGS, PCI_Config, 0x00, 0x04)	seems ok
0x000f6b50/DSDT.dsl	OperationRegion (AWK0, SystemMemory, (ECFG + 0x0232), 0x20)	suspicious
0x000f6b50/DSDT.dsl	OperationRegion (QREG, SystemMemory, (ECFG + 0x0100), 0x10)	suspicious
0x000f6b50/DSDT.dsl	OperationRegion (LPCS, SystemMemory, ECFG, 0x0400)	seems ok

ACPI rootkit scanner

Live-Demo

Removing ACPI Rootkit

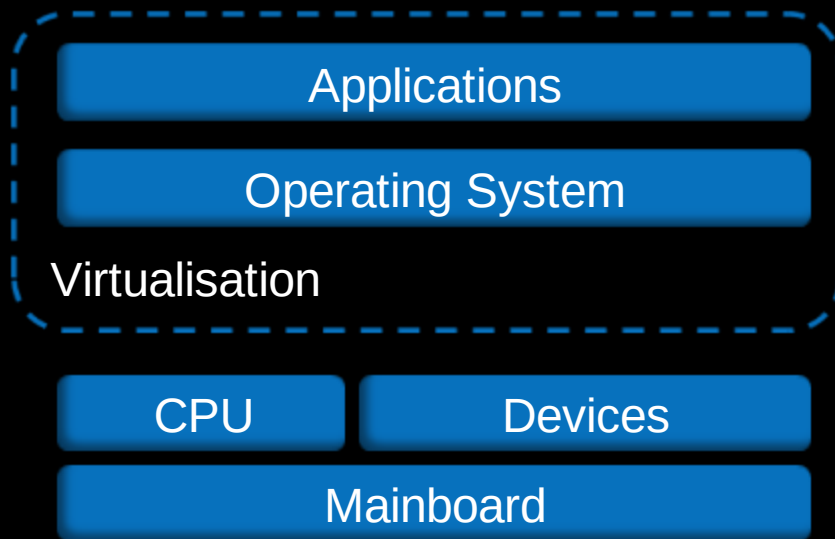


Source [6]

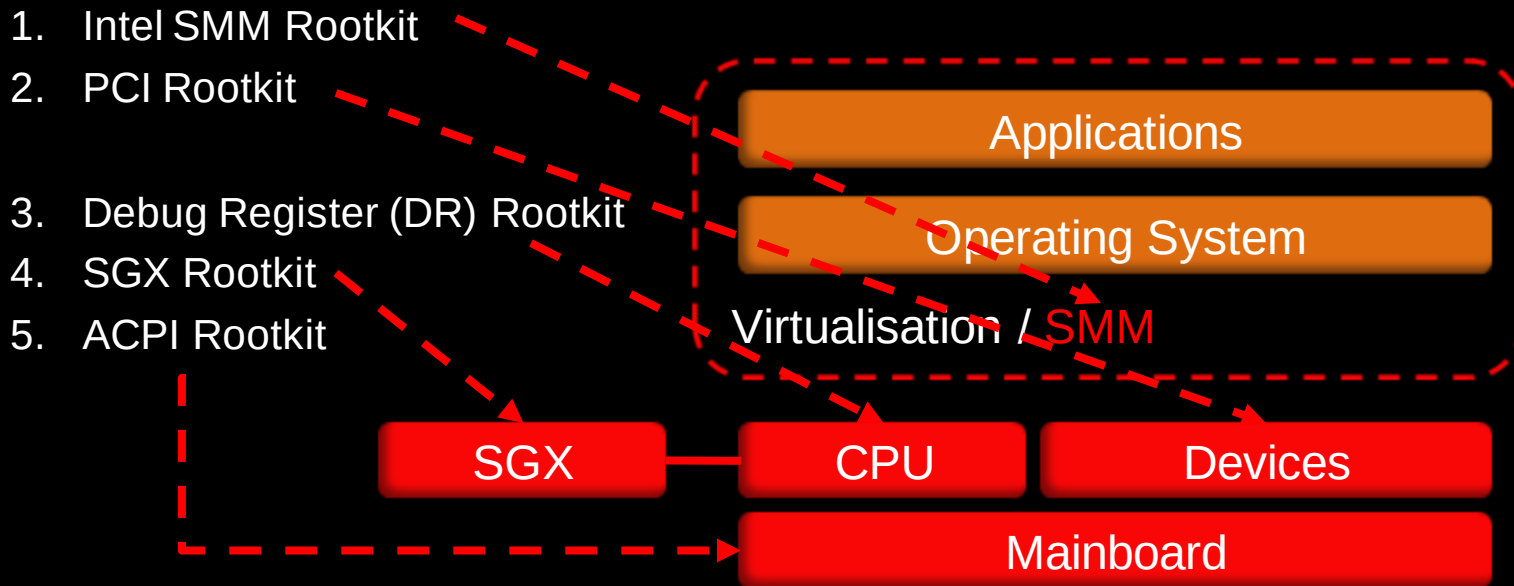


Source [7]

Advanced (Firmware) Rootkits



Advanced (Firmware) Rootkits



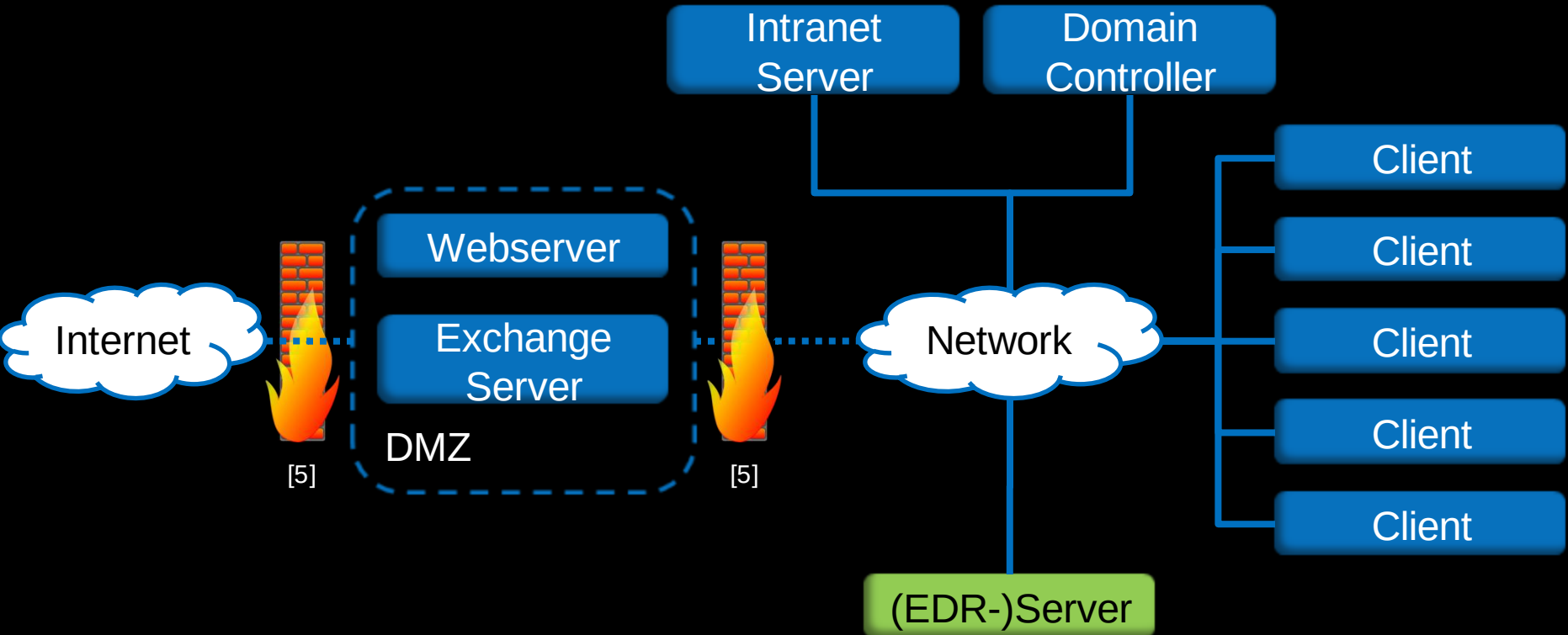
Future: Live-Forensics

- How to take a RAM-image (spontaneously)?
 - specialized software (preinstalled?)
- How servercluster 100 server?

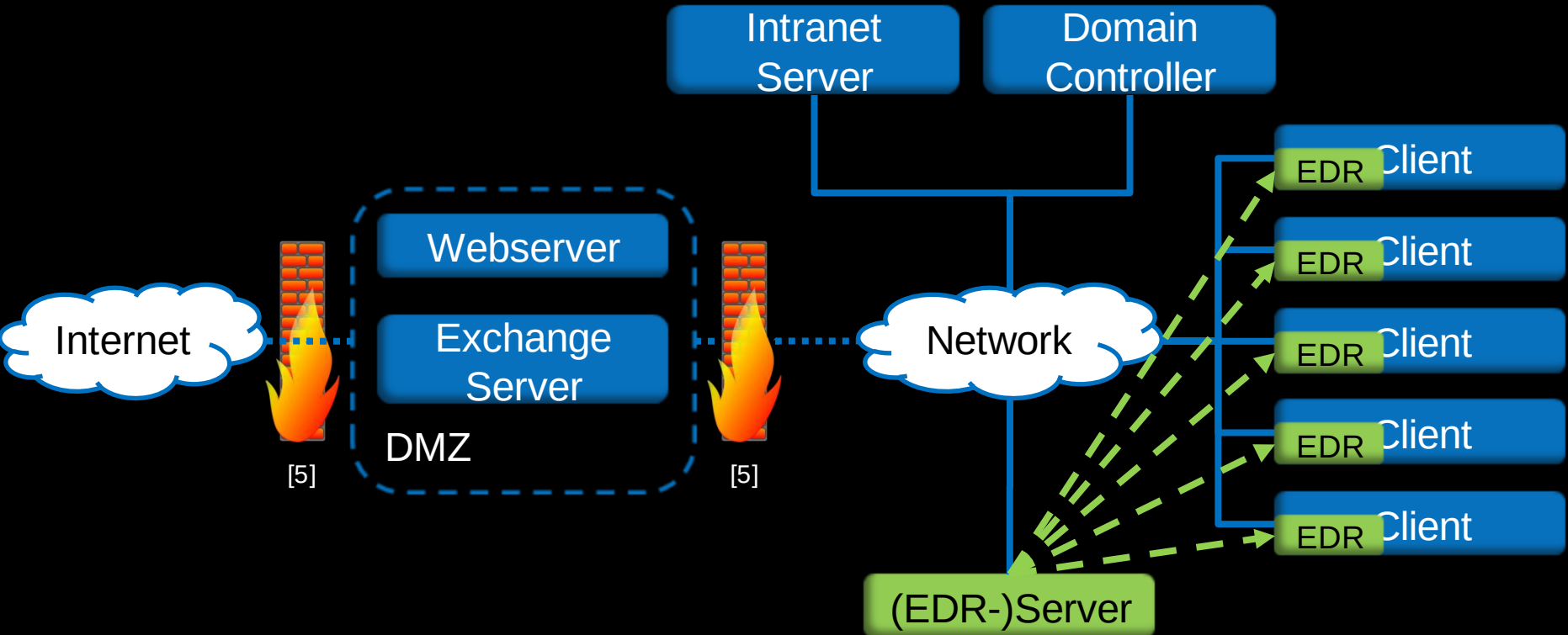
Endpoint Detection & Response (EDR)

- Digital Forensics
- Incident Response
- Client (Agent) & Server

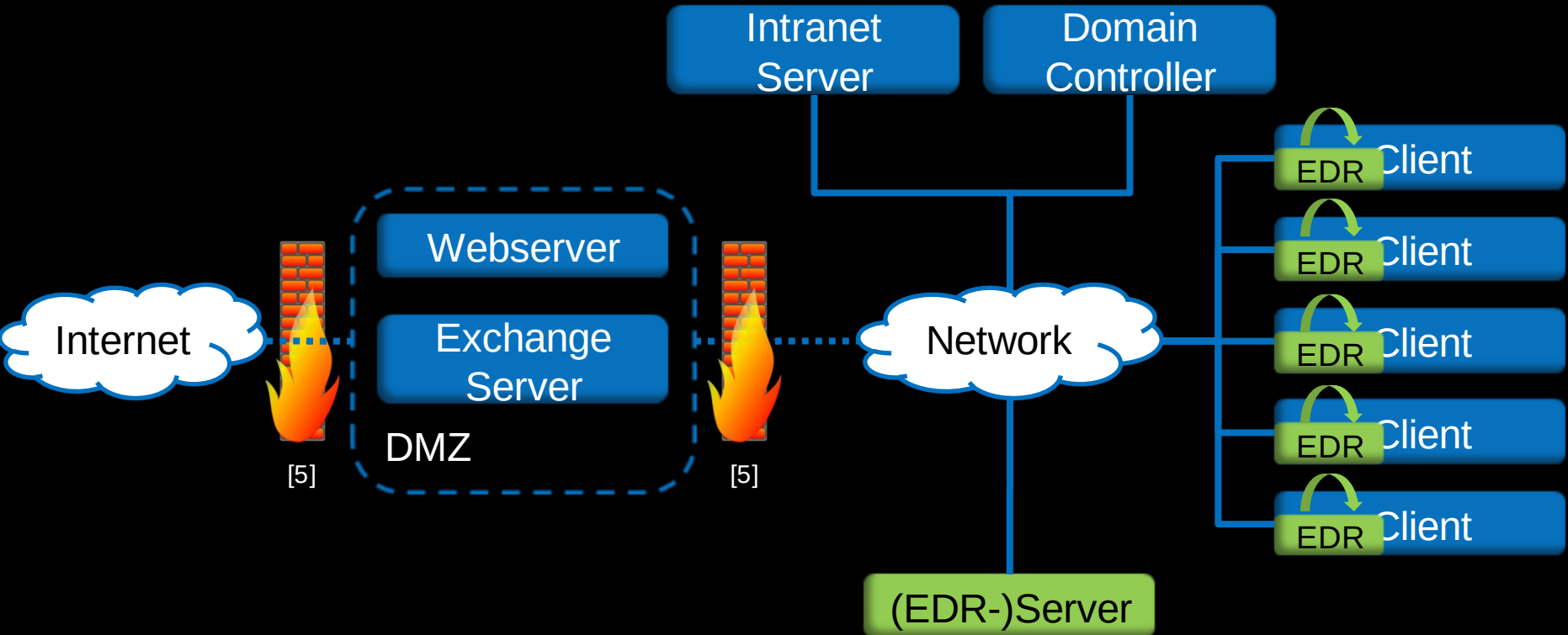
Endpoint Detection & Response (EDR)



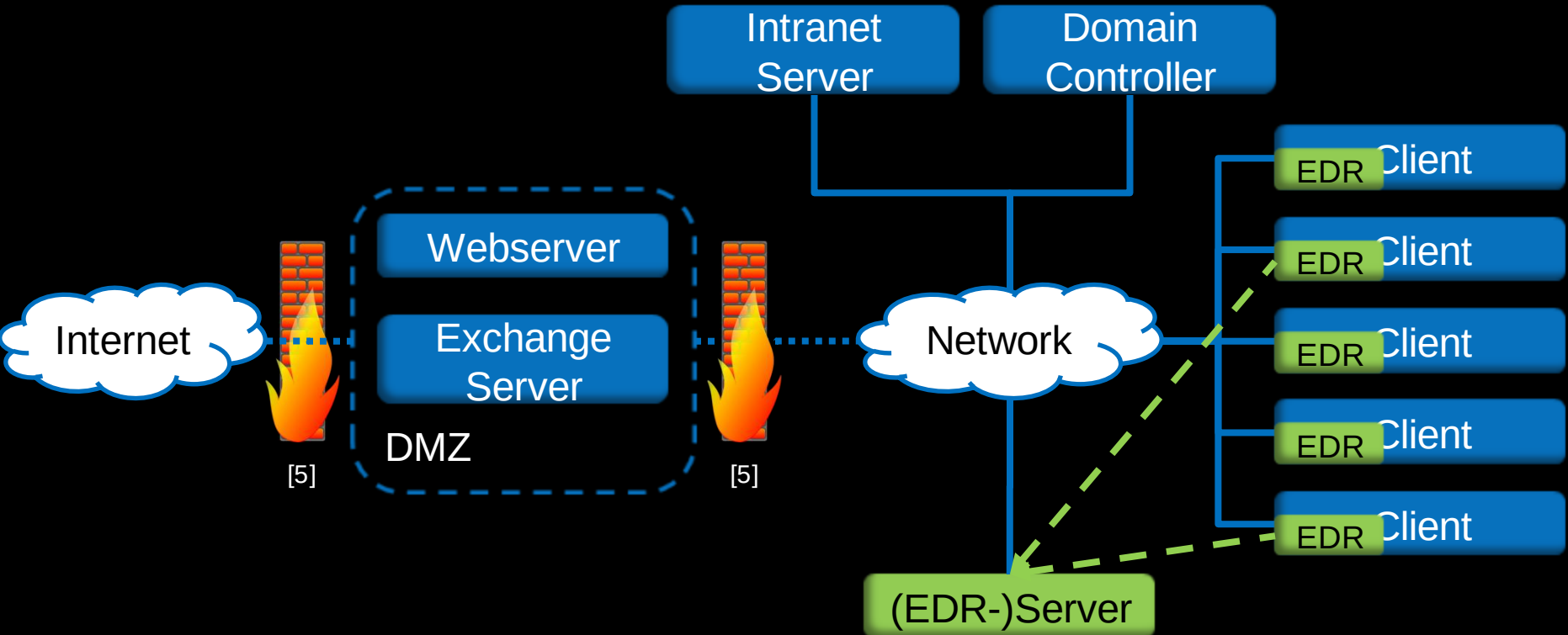
Endpoint Detection & Response (EDR)



Endpoint Detection & Response (EDR)



Endpoint Detection & Response (EDR)



velociraptor

- by: Michael Cohen (=volatility)
- open-source EDR
- Client-Server and one-shot collection scripts
- can run file-less (no installation)
- Go binary (runs on all common platforms)

velociraptor Basics

Velociraptor Response and Monitor

Search clients

MSEDGEWIN10 connected

admin

NotebookId Name Description Creation Time Modified Time Creator Collaborators

N.C6DEU42AU6UF8	Demo		2021-11-22 01:14:56 UTC	2021-11-22 01:31:20 UTC	admin	
-----------------	------	--	-------------------------	-------------------------	-------	--

Query Stats: {"RowsScanned":0,"PluginsCalled":0,"FunctionsCalled":0,"ProtocolSearch":0,"ScopeCopy":0}

Users

Uid	Gid	Name	Description	Directory	UUID	Mtime	Type
1000	513	IEUser	IEUser	C:\Users\IEUser	S-1-5-21-3461203602-4096304019-2269080069-1000	2021-08-04T16:21:27.6328729Z	local

Showing rows 1 to 1 of 1

Registry

Name	FullPath	ModTime	StartupPath
OneDrive	\HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run\OneDrive	2019-03-19T11:17:57.7796914Z	"C:\Users\IEUser\AppData\Local\Microsoft\OneDrive\OneDrive.exe /background

2021-11-22T01:36:40.337Z

Velociraptor VQL

- Basics

```
SELECT * FROM Artifact.Windows.Sys.Users()  
WHERE Name =~ "IE"
```

- Registry

```
SELECT Name, FullPath, ModTime, Data.value AS StartupPath  
FROM glob(globs='''/HKEY_CURRENT_USER/**/Run/****', accessor="registry")  
LIMIT 5
```

- Combined Queries

```
SELECT * FROM foreach(  
  row={  
    SELECT Directory FROM Artifact.Windows.Sys.Users()  
  },  
  query={  
    SELECT FullPath, Mtime FROM glob(globs=Directory+"/*", accessor="ntfs")  
  }  
)
```

Velociraptor Live-RAM Analysis

```
SELECT Pid, Name,  
       format(format="%0#x", args=PebBaseAddress) AS PebBaseAddress,  
       parse_binary(accessor="process",  
                    filename=format(format="/%v", args=Pid),  
                    profile=profile,  
                    struct="PEB",  
                    offset=PebBaseAddress).ProcessParameters.CommandLine.Buffer AS CommandLine  
FROM pslist()  
WHERE Name =~ "velociraptor" | Name =~ "Chrome"
```

Explanation:

- „FROM pslist WHERE ...“ → get all running tasks named „velociraptor“ or „chrome“
- „parse_binary“ → parse their binary from RAM
- PEB = process environment block (OS structure)
- ProcessParameters.Commandline → get commandline

Velociraptor Live-RAM Analysis

Pid	Name	PebBaseAddress	CommandLine
6600	velociraptor-v0.6.2-windows-amd64.exe	0xa4c7f79000	"C:\Users\IEUser\Downloads\velociraptor-v0.6.2-windows-amd64.exe" gui
6684	chrome.exe	0x68999df000	"C:\Program Files\Google\Chrome\Application\chrome.exe" --single-argument
6748	chrome.exe	0xc9d6b51000	"C:\Program Files\Google\Chrome\Application\chrome.exe" --type=crashpad-h url=https://clients2.google.com/cr/report --annotation=channel= --annotation=p
6880	chrome.exe	0x32a4efd000	"C:\Program Files\Google\Chrome\Application\chrome.exe" --type=gpu-proces preferences=UAAAAAAAAADgAAAYAAAAAAAAAAAAAAAAABgAIAAAAAw --mojo-platform-channel-handle=1716 /prefetch:2
6896	chrome.exe	0xdcacb7a000	"C:\Program Files\Google\Chrome\Application\chrome.exe" --type=utility --utilit
6968	chrome.exe	0x646bfd000	"C:\Program Files\Google\Chrome\Application\chrome.exe" --type=utility --utilit

velociraptor Demo

Live-Demo

Up for an adventure in Europe?



CyberSecurity Jobs

Master thesis in Digital Forensics & Incident Response / student positions:

- Contact me: michael.denzel@airbus.com

Cyber Security Incident Responder (d/f/m):

- https://ag.wd3.myworkdayjobs.com/en-US/Airbus/job/Mnchen-Area/Cyber-Security-Incident-Responder--d-f-m_JR10038421-1

Cyber Security Internship Penetration Tester (d/f/m):

- https://ag.wd3.myworkdayjobs.com/en-US/Airbus/job/Mnchen-Area/Praktikum-im-Bereich-Penetration-Testing_JR10094534

All other Airbus Cyber Security positions:

- https://www.airbus.com/careers/search-and-apply/search-for-vacancies.html?filters=filter_1_2&textsearch=Cyber

Contact our Cyber Security Recruiter!

- Cornelia Seebauer - cornelia.seebauer@airbus.com
- <https://www.linkedin.com/in/cornelia-seebauer-3159b0154>



Contact (for research questions)

Dr. Michael Denzel
md.research@mailbox.org

Sources

- [1] <https://www.blackhat.com/presentations/bh-europe-06/bh-eu-06-Heasman.pdf>
- [2] <https://github.com/mdenzel/ACPI-rootkit-scan>
- [3] <https://github.com/volatilityfoundation/volatility/wiki/Command-Reference#pslist>
- [4] <https://github.com/volatilityfoundation/volatility/wiki/Command-Reference-Mal#ldrmodules>
- [5] <https://pixabay.com/vectors/firewall-fire-wall-computer-153179/>
- [6] <https://unsplash.com/photos/1SAnrlxw5OY>
- [7] <https://unsplash.com/photos/LctoBGY6cR8>